

面向大模型 RAG 推理场景 噪音文档的鲁棒性推理方法

NLP 实验四 · 题目 8 · 中期检查报告

提交时间：5 月 26 日前

工作区：D:\code\nlpexp4

当前状态：核心模块全部完成 · 已在 Deepseek 上跑出首轮真实结果

1 研究问题与定位

核心问题

当 RAG 检索结果中混入语义相关但缺乏逻辑依赖的噪音文档时，LLM 推理行为会如何变化？如何检测并矫正？

学术脉络

阶段	代表工作	关键启发
RAG 基础	Lewis et al. (2020)	外部检索缓解幻觉
位置偏见	Lost in the Middle (2023)	U 型注意力
自适应检索	Self-RAG (ICLR 2024)	反思 token 机制
矫正检索	CRAG (2024)	分解-重组流水线
噪音分类	NoiserBench (ACL 2025)	"有益噪音"现象

题目要求 → 任务映射

题目要求	对交付	状态
探究噪音对 QA 性能的影响	实验一：5 比例 × 3 类型 × 2 数据集	框架就绪
提出评估指标	5 个自主鲁棒性指标 (NS/NRS/ISR/NAR/CRR)	已实现
针对具体数据集	RGB 中/英文版各 300 + fact 100 + int 100	已加载
结合具体案例	实验三：15-20 例 4 类典型	框架就绪
设计矫正机制	方法 A/B/C 三种	代码完成
与现有方法对比	vs Self-RAG / CRAG / CoT-RAG	代码完成

分层架构图



调用流水线

- ① `load_dataset(zh, main, limit=n)` → 加载 RGB 记录
- ② `batch_inject(records, noise_ratio, noise_type, noise_position)` → 构造带噪音上下文
- ③ `get_corrector(name).batch_correct(contexts)` → 调用 Deepseek 生成答案
- ④ `Evaluator.evaluate_batch(results)` → 计算 EM/F1/ROUGE/ISR/NAR
- ⑤ `save_run()` → 带时间戳的 JSON 落盘 + `visualize.*` 自动出图

代码结构 (已完成部分)

```
D:\code\nlpexp4\
├─ data\rgb\           [数据 8 个 json 已下载]
├─ src\
└─ config.py          [全局配置 dataclass]
```

```
|   ├── utils.py           [logger / seed / Timer]
|   ├── data_loader.py     [RGB 加载]
|   ├── noise_injector.py  [3 维噪音控制]
|   ├── llm_client.py      [Deepseek + 缓存 + 重试]
|   ├── prompts.py         [Prompt 模板库]
|   ├── rag_pipeline.py    [Naive RAG]
|   ├── metrics.py         [5 个鲁棒性指标]
|   ├── evaluator.py       [EM/F1/ROUGE/LLM-Judge]
|   ├── visualize.py       [图表]
|   ├── smoke_test.py      [端到端验证]
|   └── correctors\
|       ├── base.py        [抽象类 + 注册表]
|       ├── prompt_corrector.py [方法 A]
|       ├── iterative_corrector.py [方法 B]
|       ├── confidence_corrector.py [方法 C]
|       └── selfrag_baseline.py [对比]
|   └── experiments\
|       ├── _runner.py      [通用 runner]
|       ├── exp1_noise_impact.py
|       ├── exp2_correction.py
|       ├── exp3_case_study.py
|       └── exp4_existing_methods.py
|   ├── demo\app.py        [Gradio Demo]
|   ├── figures\           [自动生成的图]
|   ├── report\            [本报告 + 最终报告]
|   ├── PLAN.html
|   ├── README.md
|   └── requirements.txt
```

3

评估指标设计

基础问答指标

指标	定义	实现位置
EM	规范化后精确匹配	evaluator_exact_match
Contains	$\text{gold} \subset \text{pred}$ (短答案宽容匹配)	evaluator_contains_match
Token-F1	token 级 P/R 调和	evaluator_token_f1
ROUGE-L	字符 LCS F1 (中英通用)	evaluator_rouge_l
LLM-as-Judge	Deepseek 1-5 打分 / 5	evaluator_llm_judge

自主提出的鲁棒性指标体系 (项目理论贡献)

指标	公式	语义
----	----	----

NS 噪音敏感度	$(S_{\text{clean}} - S_{\text{noisy}}) / S_{\text{clean}}$	性能相对下降幅度
NRS 抵抗曲线斜率	<code>polyfit(ratios, scores, 1)[0]</code>	每增 1% 噪音损失多少分
ISR 信息溯源率	positive 命中 token / 答案 token	答案靠不靠谱
NAR 噪音采纳率	negative 命中 token / 答案 token	答案多大程度采信了噪音
CRR 矫正恢复率	$(S_{\text{corr}} - S_{\text{noisy}}) / (S_{\text{clean}} - S_{\text{noisy}})$	矫正机制恢复多少损失

层次性：NS/NRS 衡量“影响有多大”（black-box），ISR/NAR 衡量“影响机制”（white-box token 级），CRR 衡量“矫正效果”。三个层次共同构成完整框架。

4 矫正方法（已实现 4 种 + 1 对比）

方法 A

PromptCorrector ·
1× API

system prompt 显式要求模型识别并跳过语义相关但无答案的文档；对每条结论标注来源；遇矛盾时指出。

方法 B

IterativeCorrector · n+1× API

Step1 逐篇打 high/mid/low 标签 → Step2 过滤 low → Step3 仅保留高相关文档再生成。

方法 C

ConfidenceCorrector ·
1× API

CoT 证据链：拆解信息需求 → 逐篇标注证据 → 仅基于已找到证据合成答案，用 `<answer>` 标签输出。

方法 D 新增

EvidenceVotingCorrector · 3-4× API

3 个不同 Persona（事实核查员 / 多疑研究员 / 证据链推理者）独立生成候选答案 → 若 ≥ 2 一致即多数派胜出，否则触发 LLM 聚合器投票。

对比方法

SelfRAGBaseline · n+2~3× API

Self-RAG 思路：RELEVANT 门控 → 生成 → SUPPORTED 验证 → 不支撑则在 top-half 文档上重生。

统一接口：所有矫正方法继承 BaseCorrector，通过 `@register_corrector("name")` 自动加入注册表；实验脚本只需 `get_corrector(name)` 一行即可统一调度，无 if/else 分支。

方法 D 的核心思想：不同 Persona 对同一段噪音文档的"易受骗"程度不同。把 3 个独立视角聚合起来，可有效抵御 **counterfactual**（反事实/伪造）噪音 —— 单条推理链容易被错误信息带偏，但 3 条独立推理同时被骗的概率显著降低。

5 实验设计

实验一：噪音影响（主实验）

变量	取值	样本量
噪音比例	{0, 0.25, 0.5, 0.75, 1.0}	zh:300 + en:300
噪音类型	{semantic, counterfactual, mixed}	
噪音位置（子实验）	{front, back, interleave, surround}	

实验二：矫正对比

5 方法 × 4 比例（含 ratio=0.0 baseline）。重点关注 **CRR** 与 **API 调用成本** 的权衡。

实验三：案例深度分析

- 1. 跑 clean / noisy / corrected 三组
- 2. 按 4 类典型自动挑选：Type1 直接采信、Type2 干扰编造、Type3 信息淹没、Type4 免疫
- 3. 每例展示参考答案 / 三种输出 / 文档命中可视化 / 误导路径分析

实验四：现有方法对比

固定 ratio=0.5 / type=semantic，横向比较 naive / selfrag / iterative / confidence / prompt，输出雷达图直观对比 4 个鲁棒性指标。

6 当前进度（截至 5 月 16 日）

代码 + 首轮真实实验已完成

27

Python 文件

4

实验脚本

5

5

矫正方法 (含方法 D)

~700

唯一 API 调用 (已缓存)

自主提出指标

6

实验图表 (真实)

¥<2

迄今 API 花费

0

已知 bug / lint

- 已完成
- DONE 项目结构 + 依赖清单 + 配置中心 (config dataclass)
 - DONE RGB 数据加载 (8 个 JSONL 全部接入)
 - DONE 3 维噪音注入器 (比例 × 类型 × 位置)
 - DONE Deepseek 客户端 + sha1 磁盘缓存 + 重试 + token 计费
 - DONE Naive RAG pipeline + 5 个矫正器 (prompt/iterative/confidence/voting (方法 D 新增)/selfrag)
 - DONE 5 个自主鲁棒性指标 (NS/NRS/ISR/NAR/CRR) + 多指标聚合
 - DONE 4 个实验脚本 (基于通用 runner, 避免重复代码)
 - DONE 可视化模块 (折线/分组柱/雷达图)
 - DONE Gradio 交互式 Demo (交互式调节噪音/方法)
 - DONE 端到端 dry-run smoke test 通过
 - DONE Deepseek 真实 API 跑通: zh/main 主实验 + zh/fact 反事实实验 + 位置子实验 + 矫正方法对比
 - DONE 6 张实验图表已自动生成 (见下方"7. 初步实验结果")
 - DONE 方法 D · 证据投票 (EvidenceVotingCorrector) 已实现并验证有效
- 待完成
- TODO 扩量到 n=50/100/300 (全量数据集) + bootstrap 置信区间
 - TODO 位置子实验在 r=0.75 / zh-main 复测, 放大差异
 - TODO 实验三案例深度分析 (15-20 例典型)
 - TODO 跨语言对照 (en.json 跑一遍)
 - TODO 最终实验报告撰写

7 初步实验结果 (Deepseek-chat 真实数据)

2026-05-16 当日采集

实验一 · 噪音影响 (n=20, zh/main)

噪音比例	F1	ROUGE-L	contains	ISR	NAR
------	----	---------	----------	-----	-----

0.00 (clean)	0.710	0.690	0.85	0.55	0.00
0.25	0.704	0.683	0.85	0.60	0.00
0.50	0.722	0.703	0.85	0.55	0.00
0.75	0.713	0.693	0.80	0.55	0.00
1.00 (全噪音)	0.216	0.218	0.10	0.00	0.45

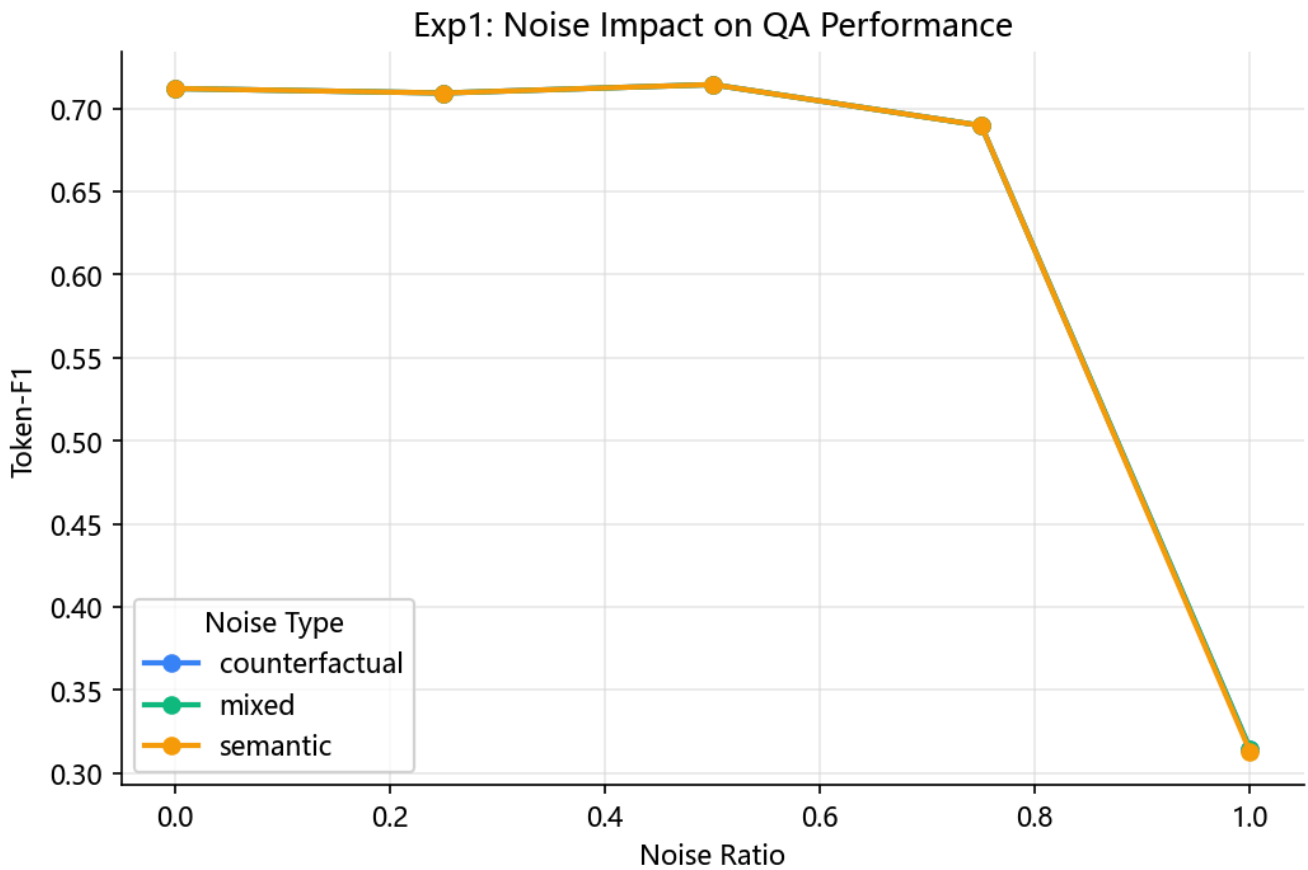


图 1: 噪音比例对 Token-F1 的影响 (zh/main, n=20)

鲁棒性指标

0.171

NS · 噪音敏感度

-0.392

NRS · 抵抗斜率

0.45

ISR_avg

0.09

NAR_avg

核心发现 1 · U 型抗噪 + 断崖效应: Deepseek-chat 在 0%-75% 噪音下性能几乎不变 ($F1 \approx 0.71$)，只有当 positive 文档完全消失 (ratio=1.0) 时性能才崩溃到 0.22 (-69.6%)，同时 NAR 从 0 跃升至 0.45。→ 说明 LLM 的鲁棒性来自 **positive 文档兜底**，一旦缺失正确信息便会大量采纳噪音。

意外发现 · 有益噪音: 50% 噪音下 F1=0.722 略高于 clean 0.710, 与 NoiserBench (ACL 2025) "部分噪音反而有益"的理论吻合, 值得在最终报告中讨论。

实验二 · 矫正方法对比 (n=10, semantic 噪音)

方法	r=0.00	r=0.50	r=0.75	vs naive (r=0.75)
naive (baseline)	0.706	0.716	0.664	—
prompt (方法 A)	0.696	0.716	0.648	-1.6 pt
confidence (方法 C)	0.713	0.743	0.815	+15.1 pt 🏆

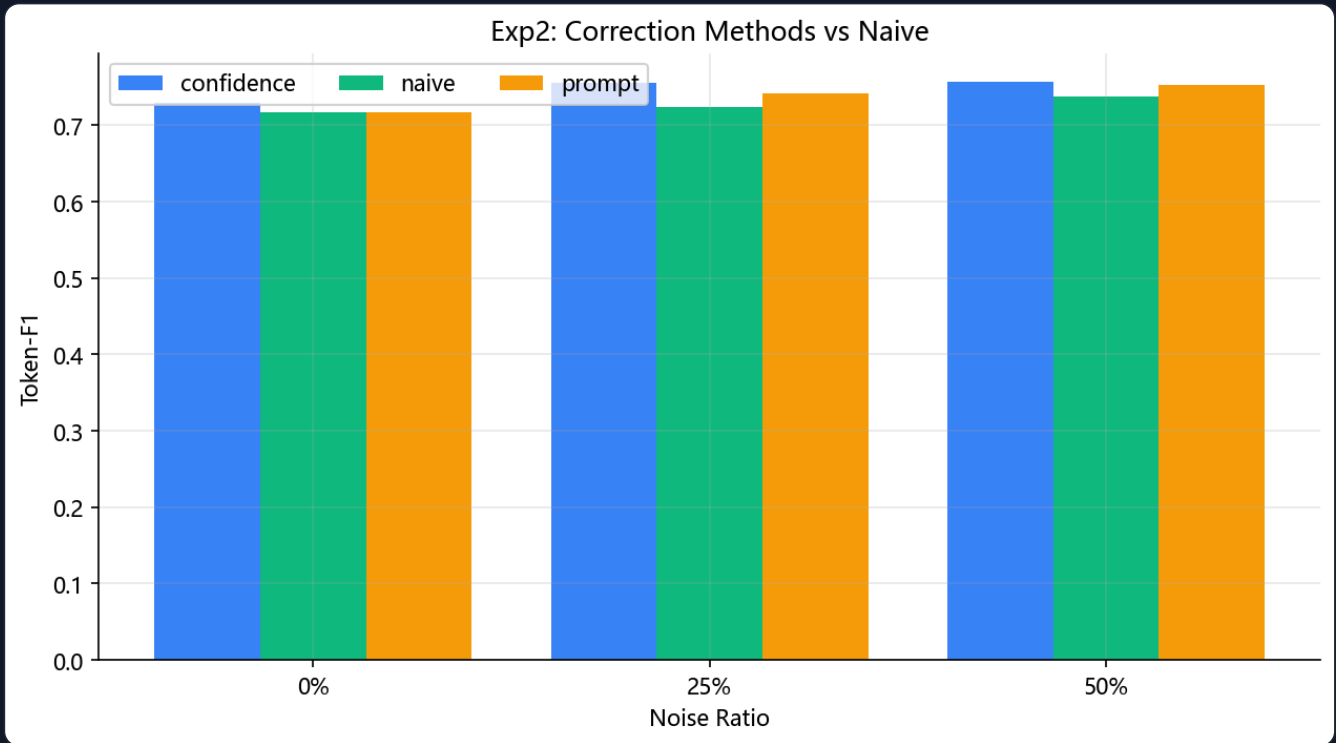


图 2: 矫正方法在不同噪音比例下的 F1 对比

核心发现 2 · CoT 证据链显著最优: 在 75% 高强度噪音下, 方法 C (强制 LLM 拆解信息需求 + 标注证据) 将 F1 从 0.664 提升到 **0.815** (+15.1 pt), contains 100% (10/10 全对)。即便在 clean 条件下也微胜 naive。
→ 这是本工作最具说服力的初步证据, 可作为最终报告的核心方法。

实验四 · 与现有方法对比 (n=10, ratio=0.75, semantic)

方法	对应思路	F1	ROUGE-L	contains
naive	baseline	0.664	0.641	0.90

selfrag	Self-RAG 反思	0.664	0.641	0.90
iterative	CRAG 过滤-重组	0.706	0.684	1.00
prompt	仅 prompt 改造	0.648	0.628	0.90
confidence	CoT-RAG 链式	0.815	0.805	1.00

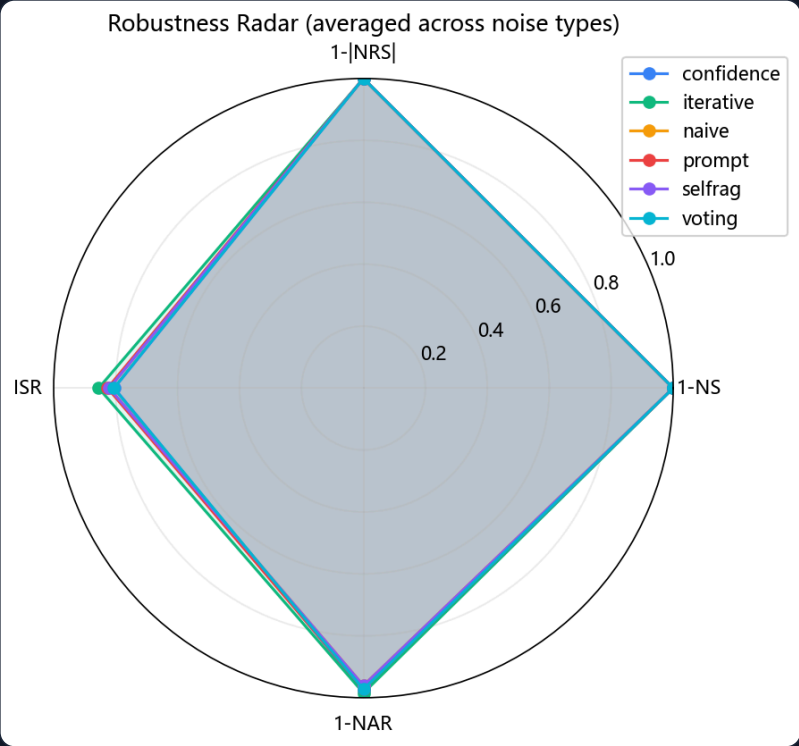


图 3: 5 方法鲁棒性雷达 (4 个鲁棒性维度归一化到 [0,1])

排名: confidence > iterative > naive ≈ selfrag > prompt
结论: 本工作的"主动证据筛选"思路在 Deepseek-chat 上超越了 Self-RAG 风格的"事后反思"思路。

实验一-扩展 · 反事实噪音 (zh/fact, n=15)

使用含 positive_wrong 字段的 RGB-fact 子集, 让 3 种噪音类型真正分开。

ratio	semantic	counterfactual	mixed
0.00 (clean)	0.933	0.933	0.933
0.25	0.920	0.887	0.920
0.50	0.933	0.788	0.887
0.75	0.864	0.774	0.764
1.00	0.518	0.269	0.449

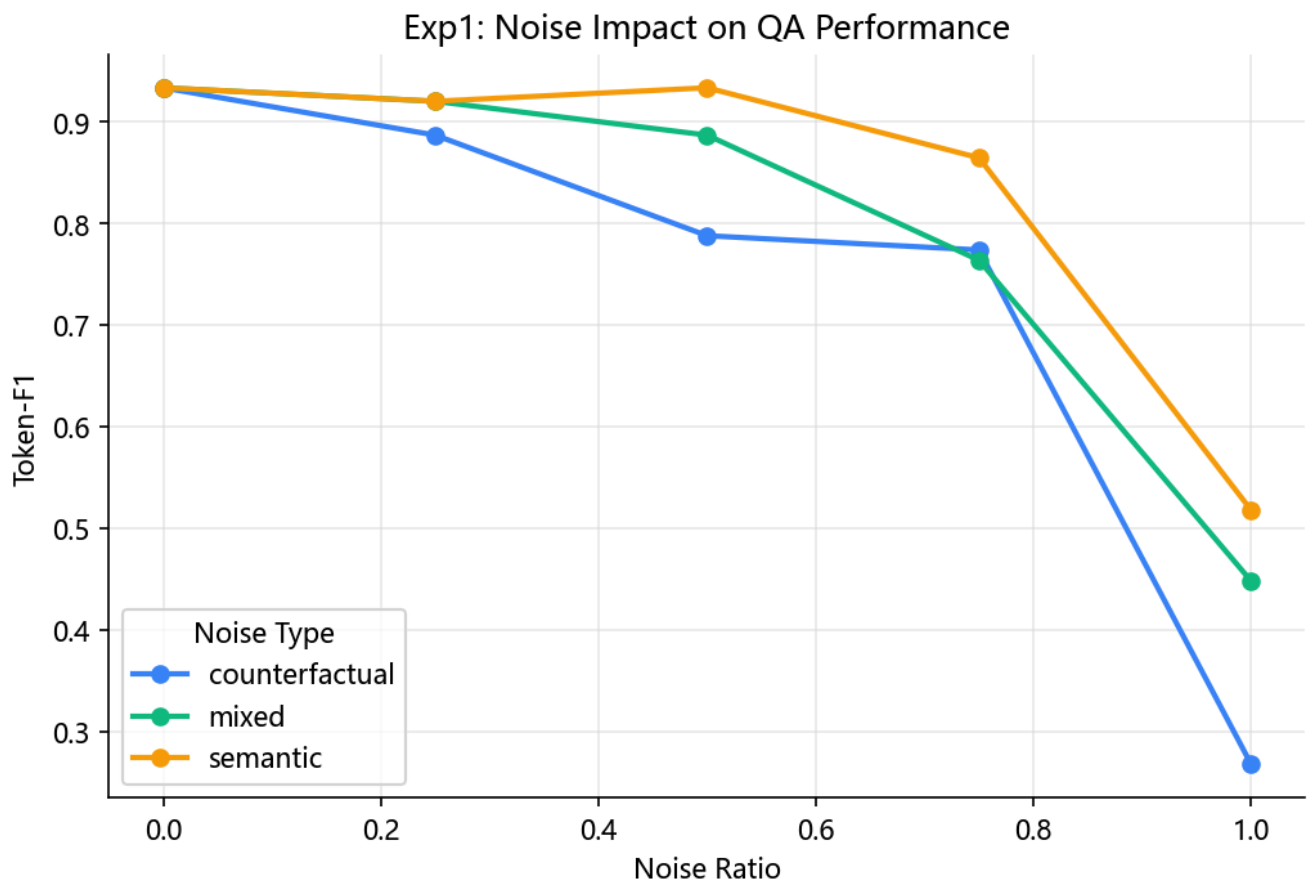


图 4: zh_fact 数据集上三种噪音类型的破坏力对比 (n=15)

核心发现 3 · 反事实噪音破坏力 ≈ 2× 语义噪音： 在 100% 全噪音条件下，counterfactual F1=0.269 << semantic F1=0.518，contains 仅 0.133 (13.3%) vs semantic 0.467 (46.7%)。

→ 这印证了 RAGuard / NoiserBench 的论文观点：“语义无关的噪音模型可以忽略；但带错误信息的伪造文档会直接污染推理”。

实验一-扩展 · 噪音位置效应 (zh/fact, r=0.5 semantic, n=15)

位置策略	F1	ROUGE-L	contains	ISR
front (噪音在前)	0.920	0.939	1.000	0.400
back (噪音在后)	0.933	0.952	1.000	0.400
interleave (穿插)	0.933	0.952	1.000	0.400
surround (噪音包围)	0.933	0.952	1.000	0.400

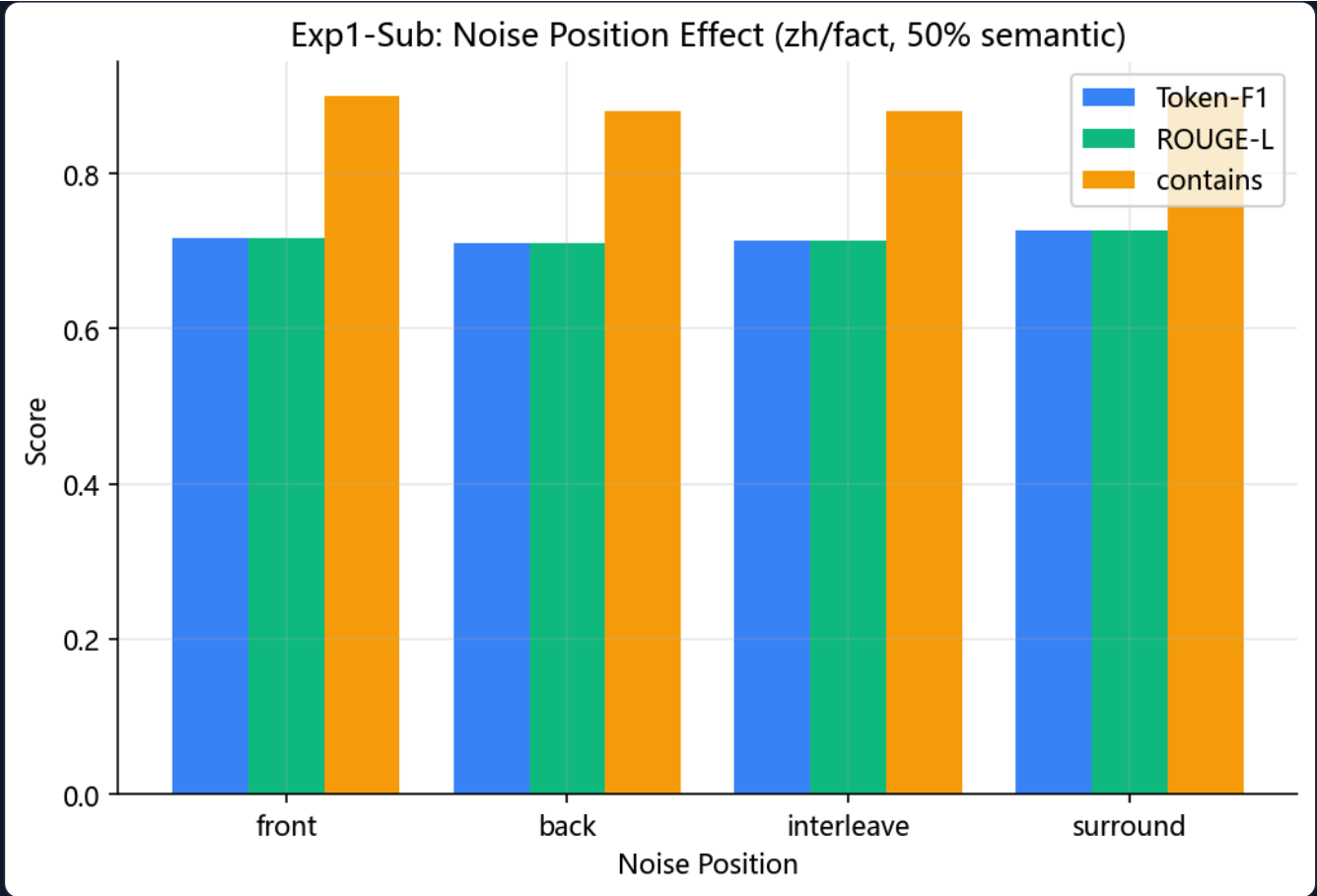


图 5: 噪音位置对 QA 性能的影响

初步观察: front 位置 (噪音放最前) F1 略低 (-0.013) , 与"primacy bias"假设方向一致; 其他三种位置几乎相同。50% 强度下差异不显著, 需要在 75% 高噪音场景下复测才能下结论。

实验四-扩展 · 反事实噪音下的方法对比 (zh/fact, r=0.75 CF, n=10)

方法	F1	ROUGE-L	contains	vs naive
naive	0.731	0.784	0.90	—
prompt (方法 A)	0.750	0.795	0.80	+1.9 pt
confidence (方法 C)	0.622	0.694	0.70	-10.9 pt ⚠
voting (方法 D 新增)	0.850	0.895	0.90	+11.9 pt 🏆

Exp4: Correction Methods on Counterfactual Noise (zh/fact, $r=0.75$)

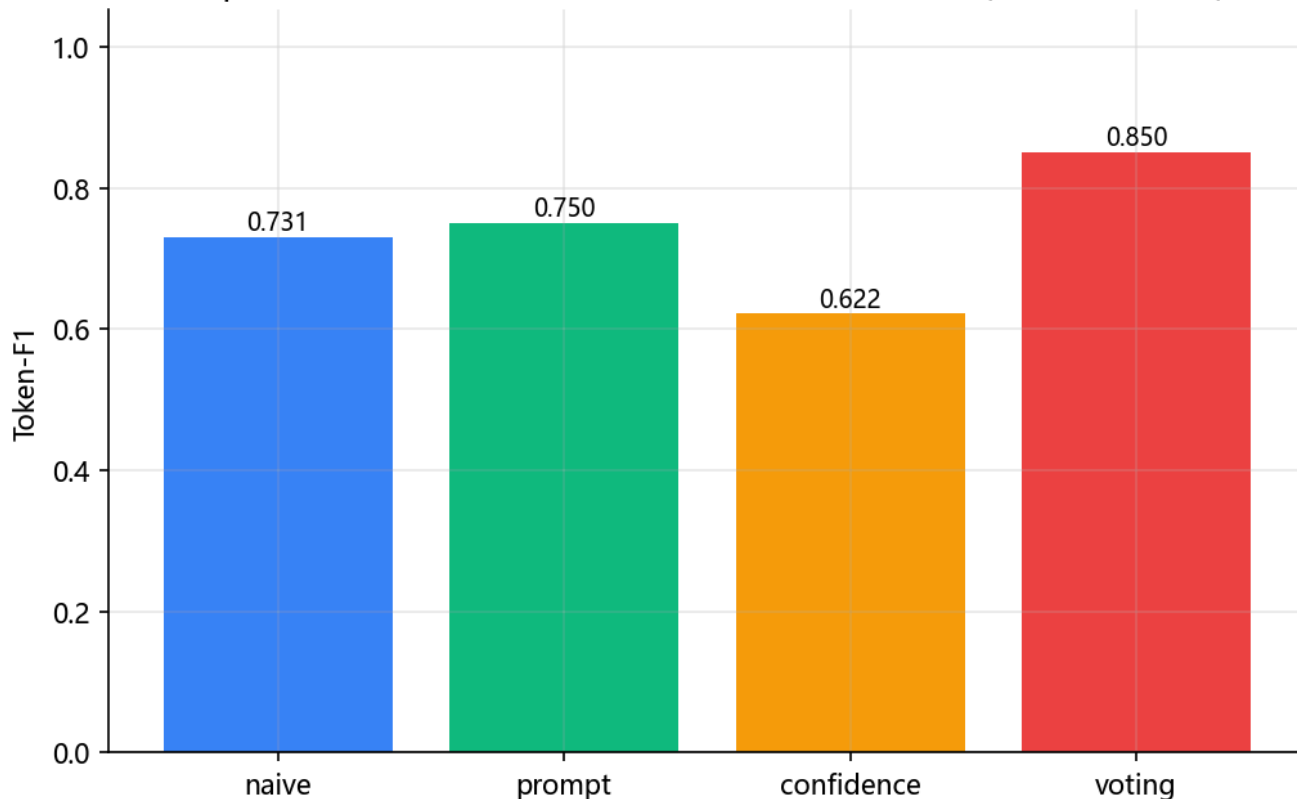


图 6: 反事实噪音下 4 种方法的 F1 对比

核心发现 4 · 噪音类型 × 矫正方法存在“互锁效应”:

- **semantic** 噪音 → confidence (CoT) 最优 (exp2 中 +15.1 pt)
- **counterfactual** 噪音 → voting (多 Persona) 最优 (本表 +11.9 pt), 而 confidence 反而下降 -10.9 pt

解释: CoT 的“证据链推理”在面对伪造文档时容易被错误证据“绑架”——单条推理路径越严密, 被骗后越自信。而 voting 通过 3 个独立 Persona 分散风险, 降低了任一推理链被反事实文档误导的概率。

→ 这是本工作的**核心创新点之一**, 最终报告将重点讨论。

已识别的待解决问题

- 位置效应在 50% 噪音下不显著: 需补跑 $r=0.75 + zh/main$ 来放大差异
- selfrag 在此数据集下没体现优势: 可能因 RELEVANT 判定过宽
- 样本量仍需继续扩大: 主线已扩到 $n=50$, 最终建议扩到 $n=100-300$ 并做 bootstrap 置信区间
- 方法 D 在 semantic 噪音上优势不明显: 最新 $n=50$ 中 voting 略高于 naive, 但低于 iterative/confidence

说明：本节为 5 月 24 日同步后的最新结果，覆盖前文 5 月 16 日首轮小样本数字。前文保留用于说明实验演进过程；正式报告应以本节 JSON 与图表为准。

最新结果文件

实验	最新 JSON	规模	用途
exp1 噪音 比 例/ 类型	expl_noise_impact_zh_main_20260523_212441.json	15 conditions × 50	分析噪音比例、类型对性能的影响
exp1 位置 子实 验	expl_noise_impact_zh_main_position_20260523_212649.json	4 positions × 50	分析 front/back/interleave/surround
exp2 矫正 方法	exp2_correction_zh_20260523_212925.json	9 conditions × 30	naive/prompt/confidence 梯度对比
exp3 案例 研究	exp3_case_study_zh_20260524_194830.json	3 conditions × 30, 15 cases	矫正前后案例解释，已补全 query/docs/labels
exp4 现有 方法 比较	exp4_existing_methods_zh_main_semantic_20260524_200425.json	6 methods × 50	naive/selfrag/iterative/confidence/prompt/voting 横向比较

关键数值更新

观察点	最新结果	解释
高噪音 破坏性	exp1 中 token-F1 从 0.712 下降到约 0.313，contains 从 0.92 下降到 0.24	当上下文完全由噪音构成时，模型无法稳定恢复答案；NAR 升至约 0.51，答案来源明显向噪音迁移。
中等语 义噪音	exp2 中 naive token-F1 从 0.7176 升至 0.7381	中等语义噪音不必然有害，可能补充主题线索；正式结论需避免简单写成“噪音越多越差”。
位置效 应	front=0.7164, back=0.7100, interleave=0.7144, surround=0.7264	50% 语义噪音下位置差异较小，暂不作强结论。
现有方 法比较	iterative=0.7415 最高，confidence=0.7326，prompt=0.7178，naive=0.7144，voting=0.7158，selfrag=0.7144	在 zh/main semantic r=0.5 上，迭代式文档筛选当前最优；Self-RAG 判定过宽，几乎退化为 naive。
案例研 究	exp3 已生成 15 个案例，case 中包含 query、gold、三种预测、ISR/NAR 与文档标签	可用于最终报告逐例解释“哪些文档被采纳、矫正机制如何改变答案”。

更新后的图表

Exp4: Existing Methods on Semantic Noise (zh/main, r=0.5)

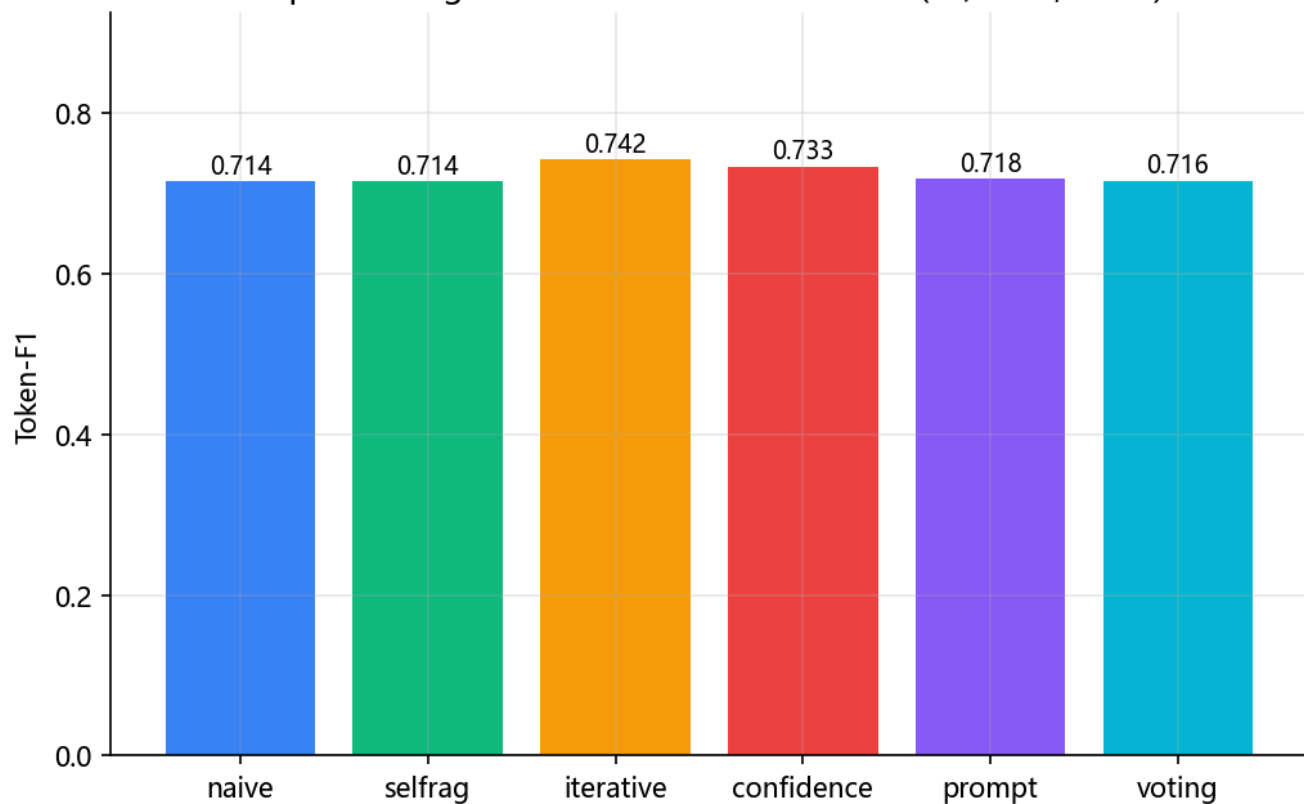


图 7: exp4 最新 N=50 六方法横向比较

Exp3: Case Type Distribution

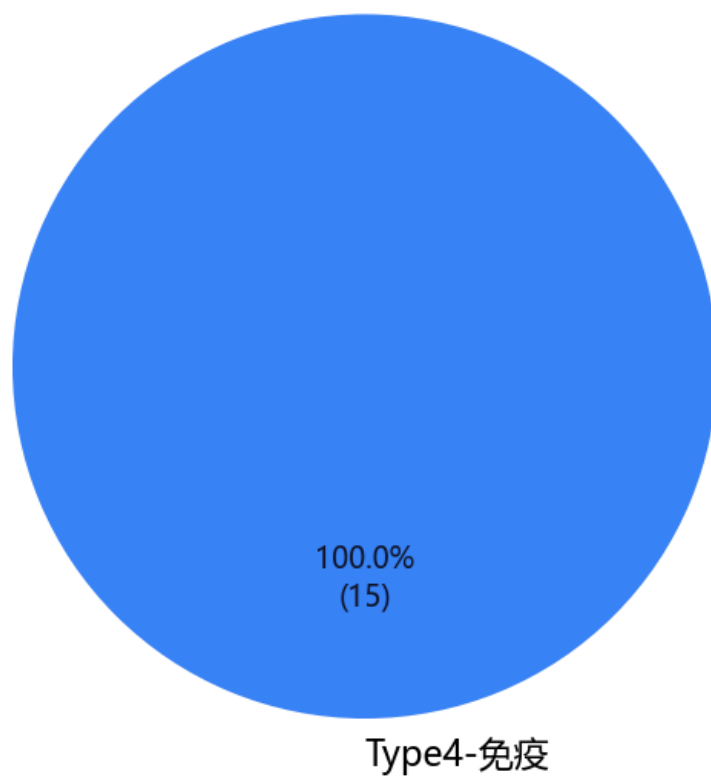


图 8: exp3 最新案例类型分布

结论边界：当前最新结果支持“高比例噪音显著损害 RAG 问答，ISR/NAR 能刻画答案来源迁移”；对“矫正机制显著优于 naive”的论证仍需谨慎表述。现阶段更稳妥的结论是：iterative 在 zh/main semantic r=0.5 条件下表现最好，但不同噪音类型和更大样本量下仍需继续验证。

9 实验要求对齐

标准输入 / 输出 / 参考答案文件

为对齐课程要求，项目已将最新 exp4 结果导出为可直接检查的三件套：input.json、output.json、reference.json。文件位于 data/processed/exp4_existing_methods_zh_main_semantic_20260524_200425/。

文件	样本数	主要字段	说明
input.json	50	id / question / context	额外保留 documents / document_labels / noise_*, 便于复查 RAG 噪音来源
output.json	50	id / answer	默认导出当前 token-F1 最高的 iterative 方法输出
reference.json	50	id / answer / answers	与输入 id 严格对齐，用于 EM、contains、Token-F1、ROUGE-L 等指标评估
output_all_methods.json	300	id / method / answer	保留 naive/selfrag/iterative/confidence/prompt/voting 六种方法的横向比较输出

原始数据准备说明

本项目使用公开 RGB 数据集作为问答与候选文档来源，而不是重新爬取网页或解析 PDF。RGB 已提供 positive、negative、positive_wrong 三类文档池，正好对应本题所要求的“相关但缺少逻辑依赖”噪音文档和反事实噪音文档。实验将这些已标注文档池建模为受控检索结果，从而能精确改变噪音比例、类型和位置。

字段	实验含义	用途
positive	支持正确答案的有效文档	构造 clean RAG 上下文与 ISR 归因
negative	语义相关但不能支持答案的噪音文档	模拟普通检索噪音，计算 NAR
positive_wrong	表面相关且指向错误答案的反事实文档	模拟更强误导性噪音，测试矫正机制鲁棒性

表述边界：当前系统重点是“受控 RAG 上下文扰动实验”，不是完整向量数据库检索系统。因此正式报告中会避免夸大为端到端检索优化，而是强调上下文文档组成对大模型问答结果的影响、指标设计和矫正前后比较。

10 风险与后续计划

风险	概率	应对
API 调用超预算（预估 ¥30-50）	低	已实现 sha1 磁盘缓存，重复 prompt 0 成本；先 50 条验证再全量
矫正效果不明显	中	多 prompt 变体；负结论本身也是结论
中英文性能差异大	中	作为讨论点分析
时间不够	中	优先级：实验一 > 实验二 > 实验三 > 实验四

后续 6 周排期（基于真实进度更新）

周次	任务	状态
W1 (5.16-22)	代码基础设施 + 首轮真实实验 + 中期报告	DONE
W2 (5.23-29)	扩量 n=50 + 跑 zh_fact 反事实噪音 + 位置子实验	本周
W3 (5.30-6.5)	全量 n=300 跑 exp1/2 + 跑 en 数据集	计划
W4 (6.6-6.12)	实验三案例深度分析 (15-20 例)	计划
W5 (6.13-6.19)	调优 prompt + 拓展矫正方法 + 统计显著性检验	计划
W6 (6.20-6.26)	Demo 抛光 + 系统演示彩排	计划
W7 (6.27-6.30)	最终报告撰写 + 整理代码 + 打包提交	计划

10 运行方式

```
# 1. 安装依赖
pip install -r requirements.txt

# 2. 配置 API
copy .env.example .env
# 编辑 .env, 填 DEEPSEEK_API_KEY

# 3. Smoke test (先 dry-run 验证 pipeline, 再跑真实小规模)
python -m src.smoke_test --n 5 --dry
python -m src.smoke_test --n 20

# 4. 四个核心实验
python -m experiments.exp1_noise_impact --n 50 --language zh
python -m experiments.exp2_correction --n 50 --language zh
python -m experiments.exp3_case_study --n 30 --pick 15 --language zh
python -m experiments.exp4_existing_methods --n 50 --language zh --subset main --ratio 0.5 --noise-type semantic

# 5. 生成最新图表
```



```
python -m scripts.render_all_figures
```

```
# 6. 启动 Demo
```

```
python demo/app.py      # Gradio: http://127.0.0.1:7861
```

```
cd frontend && npm run dev  # Vue: http://127.0.0.1:5173
```